

河狸云 × AI Kernel 对接完整方案

基于《多租户可切换场景下的 Ticket、SessionScope 与 Gateway 重设计方案》定稿 覆盖：身份打通、多租户切换、嵌入模式、独立访问模式、能力网关

一、全景：四种访问方式，一套内核协议

访问方式	用户入口	身份来源	企业切换方式
嵌入模式 (iframe)	河狸云内点”AI助手”	河狸云签发 ticket 传入 iframe	河狸云前端 postMessage 通知
嵌入模式 (新窗口)	河狸云内点”AI助手”打开新标签	河狸云签发 ticket 通过 URL 传入	AI Kernel 自己的 scope 选择器
独立访问 (有河狸云登录态)	直接打开 ai.ricent.com	跳转河狸云 /ai-auth → 自动签 ticket 跳回	AI Kernel 自己的 scope 选择器
独立访问 (未登录)	直接打开 ai.ricent.com	跳转河狸云 /ai-auth → 登录 → 签 ticket 跳回	AI Kernel 自己的 scope 选择器

四种方式共用同一套 Ticket → Session → activate-scope → Gateway 协议。

二、河狸云侧需要提供的三个东西

接口 1: Ticket 签发 (P0)

POST /api/internal/ai-ticket

河狸云内部调用（已登录用户触发），签发包含完整 identity + memberships 的 ticket。

请求：使用当前登录态，无 Body。

Ticket 内容 (JWT Payload)：

```
{
  "issuer": "beaver",
```

```
"audience": "beaver-ai-kernel",
"subject": "user:10000",

"user": {
  "user_id": "10000",
  "username": "admin",
  "display_name": "系统运维",
  "phone": "13800138000"
},

"memberships": [
  {
    "membership_id": "m_RYSGS_1583859697503438597",
    "enterprise_id": "RYSGS",
    "enterprise_name": "瑞信 | 智慧建造",
    "tenant_id": "RYSGS",
    "member_id": "1583859697503438597",
    "org_id": "RYSGS",
    "oid": "495232527520432",
    "role_type": 128,
    "roles": [{"name": "集团管理员", "group": "Enterprise"}],
    "regions": [],
    "job": "总经理,项目经理",
    "children_ids": ["RYSGS0000F"]
  },
  {
    "membership_id": "m_BLOGI_1769928738350792139",
    "enterprise_id": "BLOGI",
    "enterprise_name": "中海地产集团",
    "tenant_id": "BLOGI",
    "member_id": "1769928738350792139",
    "org_id": "BLOGI",
    "oid": "495232527520432",
    "role_type": 64,
    "roles": [{"name": "项目管理员", "group": "Project"}],
    "regions": ["华南区"],
    "job": "项目经理",
    "children_ids": []
  }
],

"default_scope": {
  "membership_id": "m_RYSGS_1583859697503438597",
  "current_module": ""
},

"access_token": "eyJ0eXAi..."
```

```
"iat": 1773676800,  
"exp": 1773677100  
}
```

字段说明：

层	字段	说明
user	user_id / username / display_name / phone	身份信息，登录后不变
memberships[]	每个企业一条	该用户在每个企业下的完整权限上下文
memberships[].membership_id	唯一标识	建议格式 m_{enterprise_id}_{member_id} , 用于 activate-scope
default_scope	签发时的当前企业	AI Kernel 初始化 session 时使用
access_token	用户在河狸云的 JWT	AI Kernel 用它代理调河狸云 API，跨企业通用

响应：

```
{  
  "code": 0,  
  "data": {  
    "ticket": "eyJhbGciOiJIUzI1NiJ9...",  
    "expires_in": 300  
  }  
}
```

参考实现：

```
@PostMapping("/api/internal/ai-ticket")  
public Result<Map<String, Object>> issueAiTicket(HttpServletRequest request) {  
    UserContext user = getCurrentUser(request);  
    String currentEntId = request.getHeader("Enterpriseid");  
    String currentMemberId = request.getHeader("Memberid");  
    String userToken = extractBearerToken(request);  
  
    // 构建 memberships (从 rc_user.enterprises)
```

```

List<Map<String, Object>> memberships = new ArrayList<>();
for (Enterprise ent : user.getEnterprises()) {
    Map<String, Object> m = new LinkedHashMap<>();
    m.put("membership_id",    "m_" + ent.getId() + "_" + ent.getMid());
    m.put("enterprise_id",    ent.getId());
    m.put("enterprise_name",  ent.getName());
    m.put("tenant_id",        ent.getTenantId());
    m.put("member_id",        ent.getMid());
    m.put("org_id",            ent.getId());
    m.put("oid",               user.getOid());
    m.put("role_type",         ent.getRoleType());
    m.put("roles",             ent.getRoles());
    m.put("regions",           ent.getRegions());
    m.put("job",               ent.getJob());
    m.put("children_ids",      ent.getChildrenIds());
    memberships.add(m);
}

// 找到当前企业的 membership_id 作为 default_scope
String defaultMembershipId = memberships.stream()
    .filter(m -> m.get("enterprise_id").equals(currentEntId))
    .map(m -> (String) m.get("membership_id"))
    .findFirst().orElse("");

long now = System.currentTimeMillis() / 1000;
Map<String, Object> claims = new LinkedHashMap<>();
claims.put("issuer", "beaver");
claims.put("audience", "beaver-ai-kernel");
claims.put("subject", "user:" + user.getUserId());
claims.put("user", Map.of(
    "user_id", user.getUserId(),
    "username", user.getUserName(),
    "display_name", user.getDisplayName(),
    "phone", user.getPhone()
));
claims.put("memberships", memberships);
claims.put("default_scope", Map.of(
    "membership_id", defaultMembershipId,
    "current_module", ""
));
claims.put("access_token", userToken);
claims.put("iat", now);
claims.put("exp", now + 300);

String ticket = Jwts.builder()
    .setClaims(claims)
    .signWith(SignatureAlgorithm.HS256, SHARED_SECRET.getBytes())

```

```
        .compact();

        return Result.ok(Map.of("ticket", ticket, "expires_in", 300));
    }
}
```

接口 2: AI 认证中转页 (P0)

GET /ai-auth?redirect={AI_Kernel回调地址}

这是一个页面，不是 API。用于“用户直接访问 AI Kernel 域名”时的身份获取。

逻辑:

用户访问 beaver.ricent.com/ai-auth?redirect=https://ai.ricent.com/auth/callback

→ 检查当前有没有河狸云登录态

→ 有登录态:

1. 调用 /api/internal/ai-ticket 签发 ticket
2. 302 跳转到 {redirect}?ticket={ticket}

→ 无登录态:

1. 跳转到河狸云登录页，带上 return_url=/ai-auth?redirect={原始redirect}
2. 用户登录成功后回到 /ai-auth
3. 此时有登录态了，走上面“有登录态”的流程

参考实现:

```
@GetMapping("/ai-auth")
public void aiAuth(
    @RequestParam String redirect,
    HttpServletRequest request,
    HttpServletResponse response
) throws Exception {
    UserContext user = getCurrentUserOrNull(request);

    if (user == null) {
        // 未登录 → 跳转登录页，登录后回来
        String returnUrl = "/ai-auth?redirect=" + URLEncoder.encode(redirect, "UTF-8");
        response.sendRedirect("/login?return_url=" + URLEncoder.encode(returnUrl, "UTF-8"));
        return;
    }
}
```

```
// 已登录 → 签发 ticket
String ticket = issueTicketForUser(request); // 复用 /ai-ticket 的逻辑

// 跳转回 AI Kernel
String sep = redirect.contains("?") ? "&" : "?";
response.sendRedirect(redirect + sep + "ticket=" + ticket);
}
```

安全要求：

- `redirect` 参数必须做白名单校验，只允许跳转到 AI Kernel 的域名
 - `ticket` 有效期 5 分钟，一次性使用
-

接口 3：Capability Gateway (P1)

POST /api/ai/gateway/execute

请求 Header (AI Kernel 根据 `active_scope` 动态注入)：

```
Authorization: Bearer {access_token}
Enterpriseid: {active_scope.enterprise_id}
Memberid: {active_scope.member_id}
Orgid: {active_scope.org_id}
Ouid: {active_scope.oid}
```

请求 Body：

```
{
  "capability_code": "issue.query",
  "params": {
    "building": "3栋",
    "status": "open",
    "page": 1,
    "page_size": 20
  },
  "trace_id": "tr_abc123"
}
```

成功响应：

```
{
  "code": 0,
```

```
"data": {
  "total": 12,
  "items": [...]
}
```

错误响应：

```
{
  "code": 4001,
  "error_type": "invalid_status",
  "message": "问题当前状态为'已关闭'，不允许重复关闭"
}
```

第一批路由表（5 个能力）：

capability_code	类型	内部接口（以实际为准）
issue.query	读	GET /api/v1/quality/issues
issue.close	写	POST /api/v1/quality/issues/{id}/close
inspection.list	读	GET /api/v1/patrol/records
acceptance.query	读	GET /api/v1/acceptance/records
project.summary	读	GET /api/v1/project/summary

统一错误码：

code	error_type	含义
0	-	成功
4001	invalid_status	业务状态不允许
4003	no_permission	无权限
4004	not_found	不存在
4010	invalid_scope	企业/组织不匹配
4040	unknown_capability	未知能力编码
5000	internal_error	内部错误

扩展方式：以后加一个 AI 场景，Java 团队在路由表加一行（10 分钟），AI 侧配一个 Action + Skill（30 分钟）。

前端配合（P0）

嵌入模式 — 河狸云前端需要做两件事：

1. “AI助手”按钮：点击时调 `/api/internal/ai-ticket` 拿 ticket，传给 AI iframe 或新窗口

```
async function openAIAssistant() {
  const resp = await request.post('/api/internal/ai-ticket');
  const { ticket } = resp.data;

  // iframe 方式
  document.getElementById('ai-iframe').src =
    `https://ai.ricent.com/chat-embed?ticket=${ticket}`;

  // 或新窗口方式
  // window.open(`https://ai.ricent.com/chat?ticket=${ticket}`);
}
```

2. 企业/模块切换时通过 `postMessage` 通知 AI iframe（完整 scope payload）：

```
function onScopeChange(newScope) {
  // newScope 来自河狸云前端切换后的状态
  const aiFrame = document.getElementById('ai-iframe');
  if (aiFrame) {
    aiFrame.contentWindow.postMessage({
      type: 'KERNEL_SCOPE_CHANGED',
      payload: {
        membership_id: newScope.membershipId,
        enterprise_id: newScope.enterpriseId,
        tenant_id: newScope.tenantId,
        member_id: newScope.memberId,
        org_id: newScope.orgId,
        ouid: newScope.ouid,
        role_type: newScope.roleType,
        current_module: newScope.currentModule,
        region_id: newScope.regionId || '',
        project_id: newScope.projectId || ''
      }
    }, 'https://ai.ricent.com');
  }
}
```

三、AI Kernel 侧的完整设计

Session 模型

```
class AISession:
    session_id: str

    # Identity — 登录后不变
    principal: Principal          # user_id, username, display_name, phone
    access_token: str             # 河狸云 JWT, 跨企业通用

    # Memberships — 用户可选的全部权限
    available_memberships: list[Membership]

    # Active Scope — 当前工作作用域, 可切换
    active_scope: ActiveScope

    # 会话状态
    created_at: datetime
    expires_at: datetime

class Membership:
    membership_id: str            # "m_RYSGS_1583859..."
    enterprise_id: str            # "RYSGS"
    enterprise_name: str          # "瑞信 | 智慧建造"
    tenant_id: str
    member_id: str
    org_id: str
    ouid: str
    role_type: int
    roles: list
    regions: list
    job: str
    children_ids: list

class ActiveScope:
    membership_id: str            # 指向哪个 membership
    enterprise_id: str
    tenant_id: str
    member_id: str
    org_id: str
    ouid: str
    role_type: int
    roles: list
```

```
regions: list
current_module: str          # "inspection" / "approval/audit"
region_id: str               # 可选
project_id: str              # 可选
```

API 端点

1. Ticket Exchange — 建立 Session

POST /api/v1/auth/exchange

Body: {"ticket": "eyJhbGciOi..."}

Response:

```
{
  "session_token": "ai_sess_abc123",
  "user": {"display_name": "系统运维"},
  "enterprises": [
    {"membership_id": "m_RYSGS...", "enterprise_name": "瑞信 | 智慧建造"},
    {"membership_id": "m_BLOGI...", "enterprise_name": "中海地产集团"}
  ],
  "active_scope": {"enterprise_name": "瑞信 | 智慧建造", "membership_id": "m_RYSGS_"},
  "expires_in": 7200
}
```

2. Activate Scope — 切换作用域

POST /api/v1/session/activate-scope

Header: Authorization: Bearer {session_token}

Body:

```
{
  "membership_id": "m_BLOGI_1769928738350792139",
  "current_module": "inspection",
  "region_id": "",
  "project_id": ""
}
```

Response:

```
{
  "active_scope": {
    "membership_id": "m_BLOGI...",
    "enterprise_id": "BLOGI",
    "enterprise_name": "中海地产集团",
    "current_module": "inspection"
  },
  "message": "已切换到中海地产集团"
```

```
}
```

3. Session Info

```
GET /api/v1/session/me
Header: Authorization: Bearer {session_token}
```

Response:

```
{
  "user": {"display_name": "系统运维"},
  "available_memberships": [...],
  "active_scope": {...}
}
```

独立访问流程

用户访问 ai.ricent.com

→ AI Kernel 前端检查有无 session_token

→ 无 session_token, 也无 URL 中的 ticket:

→ 跳转 beaver.ricent.com/ai-auth?redirect=https://ai.ricent.com/auth/callback

→ 河狸云检查登录态

→ 已登录 → 签 ticket → 跳回 ai.ricent.com/auth/callback?ticket=xxx

→ 未登录 → 河狸云登录页 → 登录 → 签 ticket → 跳回

→ AI Kernel /auth/callback 页面收到 ticket

→ 调 POST /api/v1/auth/exchange 换 session

→ 拿到 session_token + memberships 列表

→ 渲染企业选择器 + 对话界面

→ 有 URL 中的 ticket (从河狸云跳过来):

→ 调 POST /api/v1/auth/exchange 换 session

→ 直接进入对话

→ 有 session_token (非首次访问):

→ 调 GET /api/v1/session/me 验证 session 有效

→ 有效 → 直接进入对话

→ 过期 → 走上面"无 session_token"的流程

scope_proxy 动态注入

在 _execute_tool_with_events 中, 从 active_scope 构建完整的请求 Header:

```
if auth_type == "scope_proxy" and scope:
    auth_type = "proxy_headers"
```

```

auth_config = {
  "headers": {
    "Authorization": f"Bearer {session.access_token}",
    "Enterpriseid": scope.enterprise_id,
    "Memberid": scope.member_id,
    "Orgid": scope.org_id,
    "Ouid": scope.ouid,
  }
}

```

scope 是从 session.active_scope 动态构建的，切换 scope 后下一次 API 调用自动带新的 Header。

嵌入模式 postMessage 监听

```

window.addEventListener('message', async (event) => {
  if (event.origin !== 'https://beaver.ricent.com') return;

  if (event.data.type === 'KERNEL_SCOPE_CHANGED') {
    const resp = await fetch('/api/v1/session/activate-scope', {
      method: 'POST',
      headers: {
        'Authorization': `Bearer ${sessionToken}`,
        'Content-Type': 'application/json'
      },
      body: JSON.stringify(event.data.payload)
    });
    const result = await resp.json();
    // 在对话中显示"已切换到 xx 企业"
    addSystemMessage(result.message);
  }
});

```

跨企业汇总查询

当用户问“我管的所有项目里哪个问题最多”：

1. AI Kernel 从 session.available_memberships 获取全部企业列表
 2. 对每个 membership 生成临时 scope，逐个调 Gateway（每次 Header 不同）
 3. 汇总结果，生成排名
 4. evidence 记录：查了哪些 membership、每次返回什么、如何汇总
-

四、密钥管理

```
# 生成
openssl rand -hex 32

# 双方配置为环境变量
AI_TICKET_SECRET=生成的密钥
```

河狸云用它签发 ticket，AI Kernel 用它验证 ticket。通过安全渠道传递。

五、交付物与时间线

河狸云侧

#	交付物	优先级	工时
1	POST /api/internal/ai-ticket (含 identity + memberships + default_scope)	P0	1 天
2	GET /ai-auth?redirect={url} (认证中转页)	P0	0.5 天
3	AI_TICKET_SECRET 密钥	P0	5 分钟
4	前端”AI助手”入口 + postMessage 企业切换通知	P0	0.5 天
5	POST /api/ai/gateway/execute (首批 5 个能力)	P1	2 天
6	统一错误码	P1	0.5 天

AI Kernel 侧

#	交付物	依赖	工时
1	scope_proxy 动态注入 (不等河狸云)	无	3h
2	Session 模型 (identity + memberships + active_scope)	无	4h

3	POST /auth/exchange (ticket → session)	河狸云 #1	3h
4	POST /session/activate-scope	#2	2h
5	GET /session/me	#2	1h
6	/auth/callback 页面（独立访问流程）	河狸云 #2	3h
7	前端企业选择器 + postMessage 监听	#4	4h
8	对接 Gateway	河狸云 #5	2h

时间线

- 第 1 周：
[河狸云] #1 Ticket 签发 + #2 /ai-auth 中转页 + #3 密钥 + #4 前端入口
[AI 侧] #1 scope_proxy + #2 Session 模型 + #4 activate-scope + #5 /me
- 第 2 周：
[双方] 联调 Ticket → Session → 对话 → 动态身份 API 调用
[河狸云] #5 Gateway 开发
[AI 侧] #3 exchange + #6 callback + #7 前端
- 第 3 周：
[双方] Gateway 联调
[AI 侧] 现有 2 个场景升级为动态身份 + 扩展 3-4 个新场景
[双方] 全链路验证（嵌入模式 + 独立模式 + 企业切换 + 跨企业查询）

六、不需要做的事

- 不需要搭 OAuth2 / OIDC Server
- 不需要搭 SSO Server
- 不需要给 AI Kernel 建独立的用户名密码体系
- 不需要改动河狸云已有的业务 API 逻辑
- 不需要改动河狸云已有的登录流程（/ai-auth 复用现有登录）

七、验证标准

--	--	--	--

#	场景	操作	期望
1	嵌入 iframe	河狸云内点AI助手	AI 显示当前企业，查询返回该企业数据
2	嵌入切企业	河狸云顶部切企业	AI 提示”已切换到XX”，后续查询用新企业
3	嵌入新窗口	河狸云内点AI助手（新标签）	AI 显示企业选择器 + 当前企业数据
4	独立访问 已登录	直接打开 ai.ricent.com	自动跳河狸云 → 自动签 ticket → 自动回来 → 进入对话
5	独立访问 未登录	直接打开 ai.ricent.com	跳河狸云登录页 → 登录 → 自动回来 → 进入对话
6	独立切企业	AI 前端选择器切换	后续查询用新企业数据
7	跨企业查询	“我管的所有项目里哪个问题最多”	AI 遍历 memberships 汇总结果
8	无权限企业	手动构造请求切到不在 memberships 中的企业	返回 403
9	session 过期	等 session 过期后操作	提示重新登录
10	Gateway 错误码	关闭已关闭的问题	AI 展示”该问题已关闭，无法重复操作”